ESCUELA POLITECNICA NACIONAL
FACULTAD DE INGENIERIA DE SISTEMAS
ARQUITECTURA DE COMPUTADORES

Taller de Programación Literaria en Python con Emacs

Evan Matango

2026-04-30

1 Objetivos

- Familiarizar al estudiante con el uso de Python dentro de Org mode en Emacs.
- Practicar programación literaria combinando texto, matemáticas y código ejecutable.
- Usar `C-c C-c` para ejecutar bloques y `C-c '` para editar bloques en un buffer especializado.
- Resolver ejercicios sencillos de Python inspirados en temas frecuentes de cheat sheets: variables, tipos, listas, condicionales, bucles y funciones.

2 Instrucciones Generales

1. Realice todas las actividades en Linux o WSL.
2. Abra este archivo en Emacs y guárdelo con un nombre como `Taller-Python-Literario-Evan.org`.
3. Ejecute bloques de código con `C-c C-c` cuando el cursor esté dentro del bloque. Org mode ejecuta el bloque y coloca el resultado en una sección `#+RESULTS`.
4. Para editar cómodamente un bloque de Python use `C-c '`. Esto abre un buffer de edición con el modo principal apropiado para el lenguaje; al usar `C-c '` nuevamente regresa al documento Org, y con `C-x C-s` se actualiza el contenido del bloque en el archivo base.
5. En este taller, en lugar de modificar código ya terminado, usted debe **escribir la lógica de programa** dentro de los bloques propuestos.
6. Exporte el documento a PDF con `M-x org-latex-export-to-pdf` cuando haya terminado.
7. Suba al aula virtual el archivo `.org` y el `.pdf`.

3 Recursos Adicionales

Puede consultar los siguientes recursos en Internet y en el Repositorio Git de la clase

- Este trabajo
- Tutorial de Python y Java en Emacs
- Tutorial LaTeX Python e Emacs
- The only Python cheatsheet you will ever need
- Comprehensive Python Cheatsheet
- Capítulo 2 del Libro *Practical Computer Architecture with Python and ARM*
- Working with Source Blocks in Emacs

4 Cómo trabajar con bloques de Python

4.1 Flujo recomendado

1. Ubique el cursor dentro de un bloque `#+begin_src python`.
2. Presione `C-c '` para abrir el bloque en una zona de edición más cómoda con soporte del modo de Python en Emacs.
3. Escriba la lógica del programa dentro de ese buffer.
4. Guarde con `C-x C-s` para actualizar el bloque en el archivo Org.
5. Regrese con `C-c '` al documento principal.
6. Ejecute el bloque con `C-c C-c` para observar el resultado insertado por Org mode.

4.2 ¿Qué debe resolver el estudiante?

En cada actividad encontrará uno o más bloques de Python con comentarios que indican el objetivo. Su tarea consiste en:

- Leer la explicación del tutorial.
- Escribir el código solicitado dentro del bloque.
- Ejecutarlo con `C-c C-c`.
- Verificar la salida.
- Explicar brevemente lo observado cuando se le pida.

No se espera copiar código de memoria. Se espera construir soluciones pequeñas y entender la relación entre texto, código y resultados.

5 Parte 1: Tutorial guiado de Python en Org mode

Esta primera parte es de familiarización. El objetivo es practicar conceptos básicos de Python y usar repetidamente `C-c '` y `C-c C-c`.

5.1 Variables y tipos

Python permite almacenar valores en variables de diferentes tipos como enteros, flotantes y cadenas. Estos son temas introductorios comunes en hojas de referencia para principiantes.

5.1.1 Actividad 1

Escriba un programa que:

- Declare una variable entera llamada `edad`.
- Declare una variable flotante llamada `promedio`.
- Declare una cadena llamada `nombre`.
- Imprima el valor y el tipo de cada variable.

```
# Escriba aquí su programa
# Debe usar variables: edad, promedio, nombre
edad = 20
promedio = 8.75
nombre = "Evan"
print(edad , type(edad))
print(promedio, type(promedio))
print(nombre, type(nombre))
```

```
20 <class 'int'>
8.75 <class 'float'>
Evan <class 'str'>
```

Explique en 2 líneas qué diferencia observa entre `int`, `float` y `str`:

- `int` almacena números enteros sin decimales, `float` maneja números con precisión decimal, y `str` se utiliza para representar cadenas de texto o caracteres.

5.2 Operaciones y expresiones

Python permite realizar operaciones aritméticas como suma, producto, potencia y residuo.

5.2.1 Actividad 2

Escriba un programa que:

- Defina dos variables numéricas `a` y `b`.
- Imprima su suma, resta, multiplicación, división y residuo.

```
# Escriba aquí su programa
a = 5.5
b = 2.0
print(f"{a}+{b}={a+b}")
print(f"{a}-{b}={a-b}")
print(f"{a}*{b}={a*b}")
print(f"{a}/{b}={a/b}")
print(f"{a}%{b}={a%b}")
```

```
5.5+2.0=7.5
5.5-2.0=3.5
5.5*2.0=11.0
5.5/2.0=2.75
5.5%2.0=1.5
```

Escriba una línea indicando para qué sirve el operador `%`:

- El operador `%` (módulo) sirve para obtener el residuo de una división entera entre dos números.

5.3 Listas

Las listas permiten agrupar varios valores y recorrerlos. Las hojas de referencia introductorias suelen mostrar creación, acceso, modificación y recorrido de listas.

5.3.1 Actividad 3

Escriba un programa que:

- Cree una lista llamada `numeros` con cinco enteros.
- Imprima la lista completa.
- Imprima el primer y el último elemento.
- Imprima la longitud de la lista.

```
# Escriba aquí su programa
numeros = [10, 20, 30, 40, 50]
print("Lista completa:", numeros)
print("Primer elemento:", numeros[0])
print("Último elemento:", numeros[-1])
print("Longitud de la lista:", len(numeros))
```

```
Lista completa: [10, 20, 30, 40, 50]
Primer elemento: 10
Último elemento: 50
Longitud de la lista: 5
```

Explique en 1 o 2 líneas qué hace `len()`:

- La función `len()` devuelve la cantidad de elementos o ítems que contiene un objeto, en este caso, determina el tamaño total de la lista.

5.4 Bucles y condicionales

Los bucles permiten repetir acciones y los condicionales permiten tomar decisiones según una condición lógica.

5.4.1 Actividad 4

Escriba un programa que:

- Recorra los números del 1 al 10.
- Imprima cuáles son pares y cuáles son impares.

```
# Escriba aquí su programa
for i in range(1, 11):
    if i % 2 == 0:
        print(f"{i} es par")
    else:
        print(f"{i} es impar")
```

```
1 es impar
2 es par
3 es impar
4 es par
5 es impar
6 es par
7 es impar
8 es par
9 es impar
10 es par
```

Describa brevemente cómo usó `if` y el operador `%`:

- Utilicé la condición `if i % 2 == 0` para verificar si el residuo del número dividido para 2 es cero; si se cumple es par, de lo contrario (`else`) es impar.

5.5 Comprensión de listas

La comprensión de listas es una manera concisa de crear listas en Python.

5.5.1 Actividad 5

Escriba un programa que:

- Genere una lista con los números del 1 al 10.
- Genere otra lista con los cuadrados de esos números usando **list comprehension**.
- Imprima ambas listas.

```
# Escriba aquí su programa
numeros = list(range(1, 11))
cuadrados = [x**2 for x in numeros]
print("Números originales:", numeros)
print("Números al cuadrado:", cuadrados)
```

```
Números originales: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Números al cuadrado: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

Explique en 2 líneas qué ventaja tiene una **list comprehension** frente a escribir varios `append`:

- Reduce la cantidad de código y lo hace más legible al permitir la generación y transformación de listas en una sola línea expresiva, mejorando la eficiencia.

5.6 Funciones

Las funciones permiten encapsular lógica y reutilizar código; son una parte básica y frecuente en resúmenes de Python.

5.6.1 Actividad 6

Escriba un programa que:

- Defina una función `cuadrado(x)`.
- La función debe retornar el cuadrado de `x`.
- Luego pruebe la función con 3 valores e imprima los resultados.

```
# Escriba aquí su programa
def cuadrado(x):
    return x**2

print("El cuadrado de 4 es:", cuadrado(4))
print("El cuadrado de 7 es:", cuadrado(7))
print("El cuadrado de 12 es:", cuadrado(12))
```

```
El cuadrado de 4 es: 16
El cuadrado de 7 es: 49
El cuadrado de 12 es: 144
```

Escriba en 2 líneas por qué las funciones ayudan a organizar programas:

- Las funciones modularizan el código, aislando tareas específicas que pueden ser reutilizadas varias veces sin necesidad de reescribir la misma lógica, facilitando el mantenimiento.

5.7 Texto, matemáticas y código

Una ventaja de la programación literaria es documentar una idea con texto y validarla con código ejecutable en el mismo archivo.

Considere la fórmula de la suma de los primeros n números naturales:

$$S(n) = \frac{n(n+1)}{2}$$

5.7.1 Actividad 7

Escriba un programa que:

- Calcule la suma de 1 hasta `n` usando un bucle.
- Calcule la misma suma usando la fórmula.
- Compare los resultados para `n = 10`.

```
# Escriba aquí su programa
n = 10
suma_bucle = 0

for i in range(1, n + 1):
    suma_bucle += i

suma_formula = (n * (n + 1)) // 2

print("Suma calculada mediante bucle:", suma_bucle)
print("Suma calculada mediante fórmula matemática:", suma_formula)
print("¿Los resultados coinciden?:", suma_bucle == suma_formula)
```

```
Suma calculada mediante bucle: 55
Suma calculada mediante fórmula matemática: 55
¿Los resultados coinciden?: True
```

Explique en 3 líneas cómo este ejemplo muestra la idea de programación literaria:

- Permite exponer un concepto matemático mediante bloques de texto e integrarlo inmediatamente con su demostración en código. Al ejecutarlo en el mismo documento, se valida empíricamente la teoría sin separar la documentación del programa.

6 Parte 2: Resolución de problemas sencillos

En esta segunda parte ya no se explica el concepto paso a paso. Ahora debe crear los bloques de código resolviendo pequeños problemas. Se recomienda usar `C-c '` para editar cada bloque y `C-c C-c` para probar la solución.

6.1 Problema 1

Escriba un programa que lea una lista fija de calificaciones `[7, 8, 10, 9, 6]`, calcule el promedio y lo imprima.

```
# Escriba aquí su solución
calificaciones = [7, 8, 10, 9, 6]
promedio = sum(calificaciones) / len(calificaciones)
print(f"El promedio de las calificaciones es: {promedio}")
```

```
El promedio de las calificaciones es: 8.0
```

6.2 Problema 2

Escriba un programa que cuente cuántas vocales hay en la cadena `"arquitectura"`.

```
# Escriba aquí su solución
cadena = "arquitectura"
vocales = "aeiou"
contador_vocales = sum(1 for letra in cadena if letra in vocales)
print(f"La palabra '{cadena}' tiene {contador_vocales} vocales.")
```

```
La palabra 'arquitectura' tiene 6 vocales.
```

6.3 Problema 3

Escriba un programa que construya una lista con los números pares del 2 al 20.

```
# Escriba aquí su solución
pares = [x for x in range(2, 21, 2)]
print("Lista de números pares del 2 al 20:", pares)
```

```
Lista de números pares del 2 al 20: [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
```

6.4 Problema 4

Escriba una función `maximo_de_tres(a, b, c)` que retorne el mayor de tres números y pruébela con un ejemplo.

```
# Escriba aquí su solución
def maximo_de_tres(a, b, c):
    return max(a, b, c)

num1, num2, num3 = 15, 42, 8
print(f"El máximo entre {num1}, {num2} y {num3} es:", maximo_de_tres(num1, num2, num3))
```

```
El máximo entre 15, 42 y 8 es: 42
```

6.5 Problema 5

Escriba un programa que imprima la tabla de multiplicar del 5, desde `5 x 1` hasta `5 x 10`.

```
# Escriba aquí su solución
for i in range(1, 11):
    print(f"5 x {i} = {5 * i}")
```

```
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
```

6.6 Problema 6

Escriba un programa que determine si la palabra `"reconocer"` es palíndroma.

```
# Escriba aquí su solución
palabra = "reconocer"

# Comparamos la palabra original con su versión invertida
if palabra == palabra[::-1]:
    print(f"La palabra '{palabra}' SI es palíndroma.")
else:
    print(f"La palabra '{palabra}' NO es palíndroma.")
```

```
La palabra 'reconocer' SI es palíndroma.
```

7 Equipo de Trabajo

Complete la información de los integrantes del grupo e indique el líder.

Nombre	email	Rol
Evan	evan.matango@epn.edu.ec	líder

8 Verificación de Entregables [100%]

- ☒ Parte 1 completada.
- ☒ Parte 2 completada.
- ☒ Cuestionario de 10 preguntas respondido en el aula virtual.
- ☒ Uso de `C-c '` para editar bloques practicado.
- ☒ Ejecución de bloques con `C-c C-c` verificada.
- ☒ Archivo PDF generado con `M-x org-latex-export-to-pdf`.
- ☒ Entrega de archivo `.org` y `.pdf`.

8.1 Problemas con la Tarea / Comentarios

- Actividad no completada debido a: N/A. Todo se completó exitosamente.
- Dificultades encontradas con Emacs, Org mode o Python: Ninguna. El uso de `C-c '` simplificó la inserción del código dentro de los bloques sin problemas de indentación.
- Comentarios adicionales: Listo para su exportación a PDF y subida final.

9 Referencias